MACHINE LEARNING IN PHYSICS
FLOW AND DIFFUSION MODELS

HARRISON B. PROSPER

PHY6937 / PHY4936

Introduction

- Probability density (pdf) estimation and sampling from a pdf are important tasks in many fields.

- Earlier we saw how a binary classifier, $D(x)$, can be used to approximate an unknown density $p(x|1)$ given a known density $p(x|0)$ using

$$p(x|1) = p(x|0)\left[\frac{D(x)}{1-D(x)}\right]$$

- Today, we introduce two other methods normalizing flows and diffusion models for approximating and/or sampling from a *d*-dimensional density, $p(x)$.

NORMALIZING FLOWS

Change of Variables (1)

Consider two probability densities $p(x)$ and $q(y)$ that are related through the function $y = f(x)$.

Suppose that $q(y)$ is *known* and our goal is to model $p(x)$.

Since $p(x)$ and $q(y)$ are probability densities, then necessarily,

$$p(x)dx = q(y)dy$$

and, therefore,

$$p(x) = q(y)\left|\frac{dy}{dx}\right|$$

Change of Variables (2)

Generally, $y = f(x)$ is *vector*-valued with $x, y \in \mathbb{R}^d$,

In this case, the change of variables formula generalizes to

$$p(x) = q(y)\,|\det J|$$

or

$$\log p(x) = \log q(y) + \log|\det J|$$

where $J = \nabla f(x)$ is the Jacobian matrix of the transformation.

Change of Variables: Example (1)

Given

$$y = f(x) = \begin{pmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{pmatrix}$$

and

$$\nabla = \left(\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2} \right)$$

the Jacobian matrix is

$$\nabla f = \left(\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2} \right) \begin{pmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{pmatrix}$$

$$= \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{pmatrix}$$

Change of Variables: Example (2)

Given

$$y = f(x) = \begin{pmatrix} r\cos\phi \\ r\sin\phi \end{pmatrix}$$

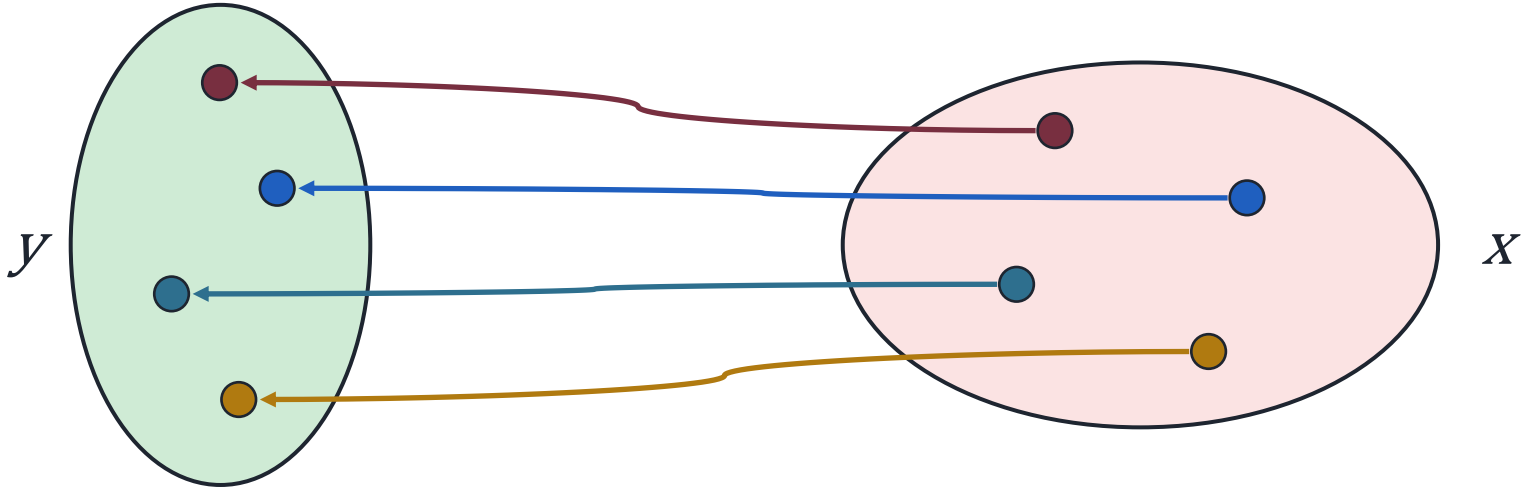
and

$$\nabla = \left(\frac{\partial}{\partial r}, \frac{\partial}{\partial \phi} \right)$$

the Jacobian matrix is

$$\nabla f = \left(\frac{\partial}{\partial r}, \frac{\partial}{\partial \phi} \right) \begin{pmatrix} r\cos\phi \\ r\sin\phi \end{pmatrix}$$
$$= \begin{pmatrix} \cos\phi & -r\sin\phi \\ \sin\phi & r\cos\phi \end{pmatrix}$$

Normalizing Flows (1)



codomain $y = f(x)$ domain

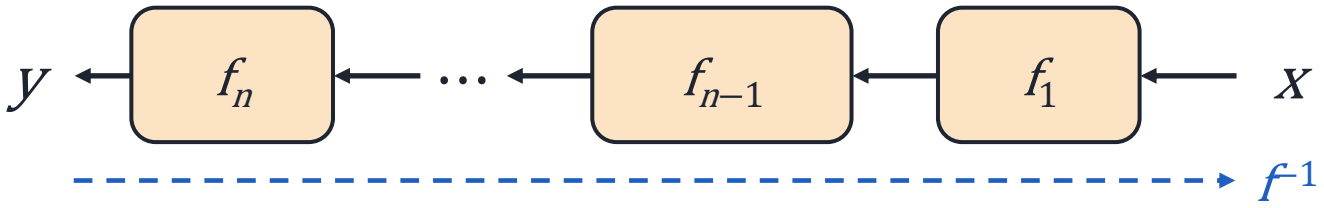
A normalizing flow is a way to approximate the function $f(x)$ by modeling the latter as a composition of bijections f_i.

$$f(x) = f_n \circ f_{n-1} \circ \cdots f_1$$

$$= f_n(f_{n-1}(\ldots f_1(x)) \ldots)$$

(A bijection is a one-to-one invertible function.)

Normalizing Flows (2)



Since the functions f_i are bijections, the inverse function $x = f^{-1}(y)$ exists.

Therefore, we can both model $p(x) = q(y)\,|\det J|$ (provided that the Jacobian is tractable) as well as sample $x \sim p(x)$ by first sampling $y \sim q(y)$ and then using $x = f^{-1}(y)$.

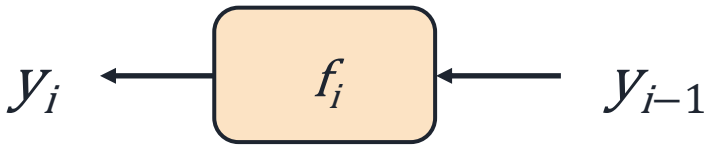
Normalizing Flows (3)

The Jacobian $|\det J|$ of $f(x) = f_n \circ f_{n-1} \cdots f_1$ is simply the product of the Jacobians of each of the functions f_i.

Therefore,

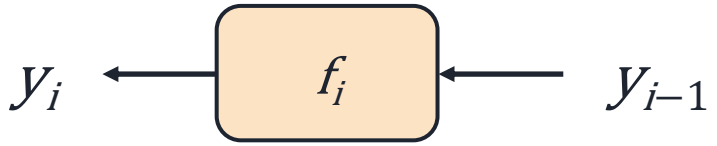
$$\log|\det \nabla f| = \sum_i \log|\det \nabla f_i|$$

The key to approximating $f(x)$ is computing the individual log Jacobians $\log|\det \nabla f_i|$ of the bijection $y_i = f_i(y_{i-1})$ where $y_0 \equiv x$.



Normalizing Flows: Bijection (1)

To simplify the calulation of the Jacobian for $y_i = f_i(y_{i-1})$,



Dinh *et al.*[1] suggest splitting the function y_i as follows

$$y_i = (y_A, y_B),$$

where y_A is of dimension m and y_B is of dimension $d - m$ with the inputs y_{i-1} split in a similar manner

$$y_{i-1} = (x_A, x_B).$$

1. Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio, *Density Estimation using Real NVP*, https://arxiv.org/abs/1605.08803

Normalizing Flows: Bijection (2)

Dinh et al.[1] suggest the following form for the bijections f_i

$$\begin{pmatrix} y_A \\ y_B \end{pmatrix} = f_i(x_A, x_B) = \begin{pmatrix} x_A \\ x_B \exp(s(x_A)) + t(x_A) \end{pmatrix},$$

where $s(x_A)$ and $t(x_A)$ are $(d-m)$-dimensional functions with $\exp$ applied *elementwise* to its vector argument.

This form yields a Jacobian that is easy to compute.

1. Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio, *Density Estimation using Real NVP*, https://arxiv.org/abs/1605.08803

Normalizing Flows: Jacobian

Writing ∇ as $(\nabla_{x_A}, \nabla_{x_B})$ and applying it to

$$\begin{pmatrix} y_A \\ y_B \end{pmatrix} = f_i(x_A, x_B) = \begin{pmatrix} x_A \\ x_B \exp(s(x_A)) + t(x_A) \end{pmatrix},$$

yields the Jacobian matrix

$$\begin{pmatrix} \nabla_{x_A} y_A & \nabla_{x_B} y_A \\ \nabla_{x_A} y_B & \nabla_{x_B} y_B \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ \nabla_{x_A} y_B & \exp(s(x_A)) \end{pmatrix},$$

with $s(x_A) = s_1(x_A), \cdots, s_{d-m}(x_A)$. The determinant is

$$\exp\left(\sum_{j=1}^{d-m} s_j(x_A)\right)$$

and hence $\log|\det J_i| = \sum_{j=1}^{d-m} s_j(x_A)$.

Normalizing Flows: Training

- Data

$$x_i \sim p(x)$$

- Loss function

$$L(\theta) = -\log p(x)$$
$$= -\left[\log q\left(f(x)\right) + \log|\det J|\right]$$

- $q(y = f(x))$ is usually chosen to be a zero mean, unit variance, diagonal d-dimensional normal.

- The unknown functions $s(x_A)$ and $t(x_A)$ are modeled with neural networks.

DIFFUSION MODELS

Diffusion Model (1)

Like a normalizing flow, a diffusion model maps a point y to a point x, where $x, y \in \mathbb{R}^d$.

One way to do this is by solving the 1st order ordinary differential equation[1, 2]

$$\frac{dx}{dt} = G(t, x)$$

where

$$G(t, x) = \lambda(t)\, x + \mu(t)\, q(t, x)$$

1. Cheng Lu†, Yuhao Zhou†, Fan Bao†, Jianfei Chen†, Chongxuan Li‡, Jun Zhu, DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps*, arXiv:2206.00927v3, 2022.
2. Yanfang Lui, Minglei Yang, Zezhong Zhang, Feng Bao, Yanzhao Cao, and Guannan Zhang, Diffusion-Model-Assisted Supervised Learning of Generative Models for Density Estimation, arXiv:2310.14458v1, 2023

Diffusion Model (2)

The functions $\lambda_t \equiv \lambda(t)$ and $\mu_t \equiv \mu(t)$ are given by

$$\lambda(t) = \frac{d \log \sigma_t}{dt}, \qquad \mu(t) = \alpha_t \frac{d \log \alpha_t/\sigma_t}{dt}$$

where a typical choices are for α_t and σ_t are

$\alpha_t \equiv \alpha(t) = 1 - t$ and $\sigma_t = \sigma(t) = \sigma_0 + (1 - \sigma_0)t$.

The conditional density $p(x\,|x_0)$ is given by

$$p(x_t\,|x_0) = A \exp\left(-\frac{1}{2} z_t^2\right), \qquad z_t \equiv z(t) = \frac{x - \alpha_t x_0}{\sigma_t}$$

where $z(t)$ is a d-dimensional vector.

Diffusion Model (3)

The main difficulty in solving the ODE[1,2]

$$\frac{dx}{dt} = \lambda_t x + \mu_t q(t, x)$$

from $t = 1$ to $t = 0$ is approximating the quantity

$$q(t, x) = \int_{\mathbb{R}^d} x_0 \, \frac{p(x \,|x_0)}{p(x)} \, p(x_0) dx_0$$

where $p(x) = \int_{\mathbb{R}^d} p(x \,|x_0) \, p(x_0) dx_0$ and $p(x_0)$ is the target density.

Diffusion Model (4)

Given a sample $\left\{x_0^{(i)}\right\}_{i=1}^{K}$ from the target density $p(x_0)$, typically from a simulation, the integral

$$q(t,x) = \frac{1}{p(x)} \int_{\mathbb{R}^d} x_0 \, p(x \,|x_0) \, p(x_0) dx_0$$

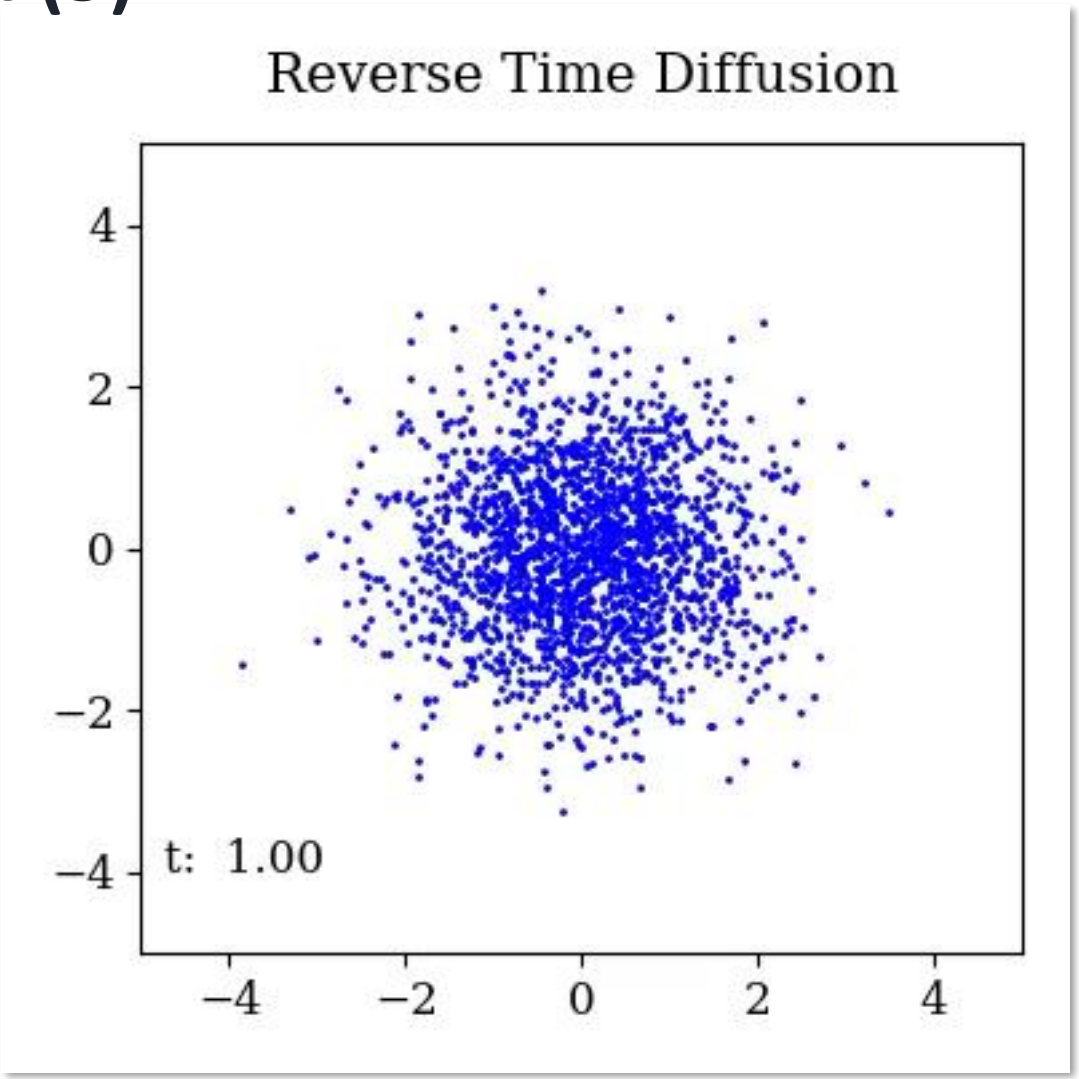
can be approximated using

$$q(t,x) \approx \sum_{n=1}^{N} x_0^{(n)} \, \text{softmax}(-u_n + u_{min})$$

where $u_n = \left[\left(x - \alpha_t x_0^{(n)}\right)/\sigma_t\right]/2$

and $u_{min} = \min(u_1, \cdots, u_N)$

Diffusion Model (5)



Diffusion Model

Advantages

- One significant advantage of this diffusion model is that the accuracy of the mapping from $x(t = 1)$ to $x(t = 0)$ depends solely on the accuracy of the numerical ODE solver and the sample size K used in approximating the integrals.
- The solution for multiple starting points can be trivially parallelized.

Disadvantages

- Solving an integro-differential equation can be computational demanding for high-dimensional problems.

Summary

- Finding good approximations to multidimensional probability densities is an important task in many fields.

- Normalizing flows have been shown to yield satisfactory results, though they may not scale well to very high-dimensional problems.

- Diffusion models can be used to map a point sampled from a Gaussian to another sampled from the desired target density. We briefly described a method that "simply" requires solving an ODE.

APPENDIX

Normalizing Flows: Pseudocode

The pseudocode that follows is inspired by François Fleuret's excellent course on deep learning https://fleuret.org/dlc/, where the quantities s and t, which are intrinsically $(d-m)$-dimensional functions, are modeled with d-dimensional tensors by using a mask, b.

Normalizing Flows: $\boldsymbol{y = f_i(x)}$

Given $b = [1, 0]$ # dim(1) = m, dim(0) = $d - m$

and $M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

$\boldsymbol{y = f(x)}$:

$x = xM$ # $x = [x_B, x_A] \rightarrow x = [x_A, x_B]$

$s = S(b \odot x)$ # dim(s) = d

$t = T(b \odot x)$ # dim(t) = d

$s' = (1 - b) \odot s$ # $s' = [0, s_B]$

$\log J = \mathrm{sum}(s')$

$y = b \odot x \ + (1 - b) \odot [x \odot \exp(s) + t]$

return $y, \log J$

François Fleuret, https://fleuret.org/dlc/

Normalizing Flows: $\boldsymbol{x = f_i^{-1}(y)}$

Given $b = [1, 0]$ # dim(1) = m, dim(0) = d−m

and $M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

$\boldsymbol{x = f^{-1}(y)}$:

$y = [y_A, y_B]$

$s = S(b \odot y)$ # dim(s) = d

$t = T(b \odot y)$ # dim(t) = d

$x = b \odot y + (1 - b) \odot (y - t) \odot \exp(-s)$

$x = xM$ # $x = [x_A, x_B] \rightarrow x = [x_B, x_A]$

return x

François Fleuret, https://fleuret.org/dlc/